SAFIR · TECHNICAL DOCUMENTATION

Sprachgestützte Assistenz für Informationserfassung in der Rettungskette

Version **2.1** · Stand: **April 2026** · Auftraggeber: **CGI Deutschland** · Zielgruppe: **Bundeswehr Sanitätsdienst**

1. Executive Summary

SAFIR ist ein zweigeteiltes, netzbasiertes Dokumentationssystem für den Bundeswehr-Sanitätsdienst entlang der **Rettungskette** (Phase 0 bis Role 4). Ein **Feldgerät** (NVIDIA Jetson Orin Nano, getragen vom Sanitäter) nimmt Sprachdiktate auf, transkribiert sie lokal mit Whisper, segmentiert mehrere Patienten pro Diktat mittels Gemma 3 4B und schreibt eine eindeutige Patient-ID auf **MIFARE-Classic RFID-Karten**. Alle Daten werden über ein **WireGuard-Mesh (Tailscale)** verschlüsselt an die **Leitstelle** (Microsoft Surface) gespiegelt.

100% ON-DEVICE KI	**~30s** E2E-LATENZ
E2E VERSCHLÜSSELT	**DE** DEUTSCHE SPRACHE

Kern-Differenzierung

- **Edge-AI:** Modelle laufen auf dem 7.4-GB-Jetson — keine Abhängigkeit von Cloud oder Internet
- **Multi-Patient pro Diktat:** ~10-minütiger freier Befund wird automatisch in strukturierte Patient-Records segmentiert
- **Physische Identifikation via RFID:** Verwundete gehen mit ihrer Karte entlang der Rettungskette
- **Halluzinations-Konservativ:** Das System vergisst lieber einen Patienten als einen zu erfinden
- **Deutsche Sprachverarbeitung:** Whisper large-v3-turbo + Gemma 3 mit deutsch-spezifischen Post-Processing-Regeln

2. Architektur-Übersicht

Zwei-Geräte-Topologie

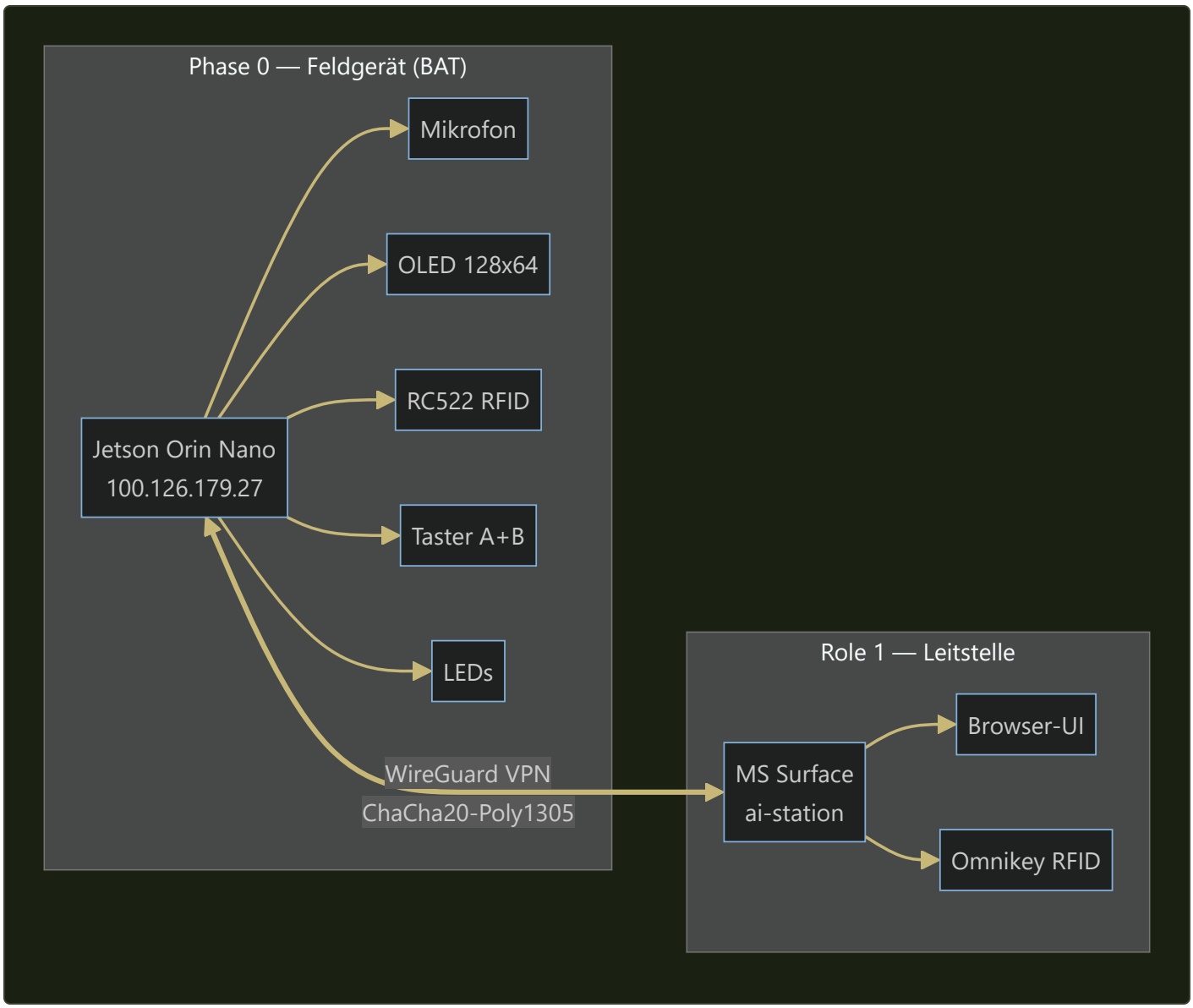


Abb. 1 — Geräte-Topologie mit verschlüsseltem WireGuard-Link

Datenfluss: Sprachbefund → Patientenrecord

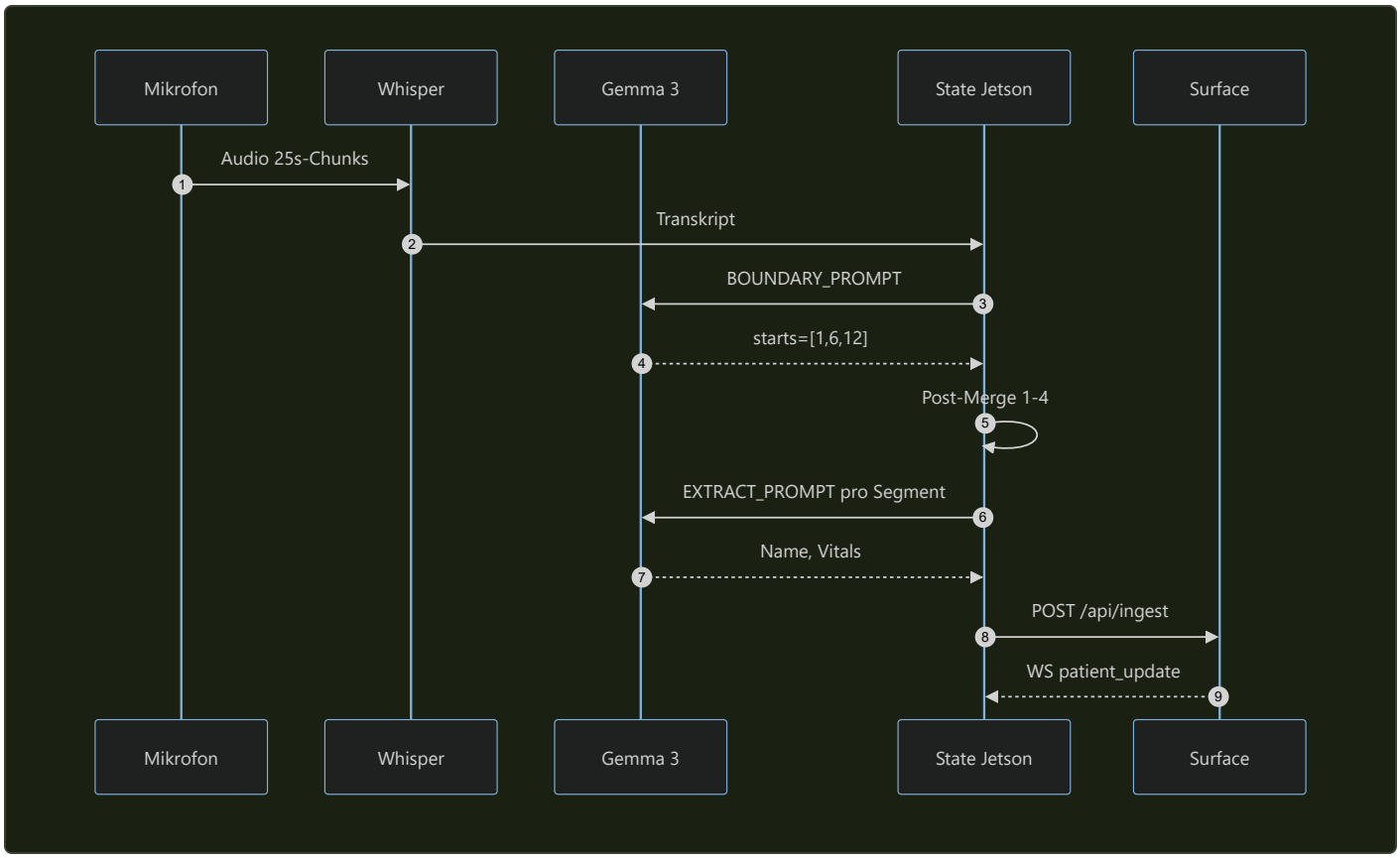


Abb. 2 — Pipeline vom Audio bis zum synchronisierten Patient

Layer-Stack

```
┌──────────────────────────────────────────────────────────┐
│ Jetson Orin Nano                                         │
├──────────────────────────────────────────────────────────┤
│ UI Layer      │ HTML Template (Jinja2) + Vanilla JS + WS │
│ API Layer     │ FastAPI (uvicorn) auf Port 8080          │
│ Business      │ app.py — Segmenter, Extract, Sync, OLED  │
│ KI Layer      │ Whisper (GPU) · Gemma 3 (GPU) · Vosk (CPU) │
│               │ · Piper TTS (CPU)                        │
│ Hardware      │ RC522 (SPI) · OLED (I²C) · GPIO · LED    │
│ OS            │ NVIDIA JetPack 5.x + CUDA 12.6           │
└──────────────────────────────────────────────────────────┘
```

3. Hardware

Feldgerät: NVIDIA Jetson Orin Nano Super

KOMPONENTE	SPEZIFIKATION
SoC	NVIDIA Orin · 6× ARM Cortex-A78AE @ 1.5 GHz · 1024 CUDA Cores (Ampere) · 32 Tensor Cores
Unified Memory	7.4 GB LPDDR5 (shared CPU+GPU)
Storage	256 GB NVMe SSD
Leistungsaufnahme	7 W (idle) · 15 W (Volllast)
Netzwerk	WiFi 6 + Ethernet Gigabit
Betrieb	Powerbank 20 000 mAh / 15 V / 65 W (~20 h Demo-Tag)

Peripherie

GERÄT	INTERFACE	FUNKTION
OLED SSD1306	I^2C Bus 7	128×64 monochrom, 4 Info-Seiten
RC522 RFID	SPI	13.56 MHz, MIFARE Classic 1K
2 Taster	GPIO Pin 11 + 26	Rot: OLED-Nav · Grün: Aufnahme
3 LEDs	GPIO	Status / Aufnahme / Fehler
Mikrofon	USB	Freisprech-Mikro
Audio-Out	USB + HDA	Parallel Headset + Lautsprecher

Leitstelle: Microsoft Surface

KOMPONENTE	SPEZIFIKATION
CPU	Intel Core i7
GPU	NVIDIA RTX 4060 (8 GB VRAM)
RAM	16 GB
OS	Windows 11
RFID-Leser	HID Omnikey USB

RFID-Karten (MIFARE Classic 1K)

```
Block 0      | Manufacturer (UID, read-only)
Block 4      | SAFIR Magic-Bytes + Version + Zeitstempel
Block 5      | Patient-ID (16 Byte)
Block 6      | Triage, Alter, Geschlecht, Blutdruck, Puls,
             | Atmung, SpO2, GCS
Block 8-9    | Name (32 Byte UTF-8)
Block 10     | Rang (reserviert)
Block 3,7,11 | Sektor-Trailer — Keys, nicht von SAFIR geschrieben
```

MIFARE CLASSIC CRYPTO1 IST NICHT KRYPTOGRAPHISCH SICHER

SAFIR behandelt die Karte daher als reine ID-Referenz. Sensitive Daten (Transkripte, freitextliche Verletzungen) landen NICHT auf der Karte, sondern nur im verschlüsselten Backend-Store.

4. Software-Stack

KI-Modelle

MODELL	TYP	GRÖSSE	HARDWARE	ZWECK
Whisper large-v3-turbo	ASR	~1.2 GB	Jetson GPU	Transkription (DE)
Gemma 3 4B (Q4_K_M)	LLM	~4.3 GB	Jetson GPU	Segmentierung + Extraktion
Vosk (de-klein)	Command ASR	~80 MB	Jetson CPU	Voice-Commands (~15ms)
Piper (Thorsten)	TTS	~65 MB	Jetson CPU	Sprachausgabe

Python-Framework

- **FastAPI** + **uvicorn** — Web-Framework auf beiden Geräten
- **Jinja2** — Template-Engine (HTML-Seiten)
- **WebSockets** — Live-Updates zwischen Geräten und Browser
- **Ollama** — lokaler LLM-Runtime für Gemma
- **whisper.cpp** + **whisper-cpp-python** — GPU-accelerated Whisper
- **sounddevice** + **numpy** — Audio I/O
- **python-docx**, **reportlab** — Export-Formate

- **pyscard** (nur Surface) — HID Omnikey-Zugang
- **httpx** — HTTP-Client für Sync

5. Datenmodell

Das zentrale Modell ist `PATIENT_SCHEMA` in `shared/models.py`.

```
{
  "patient_id": "PAT-XXXXXXXX",
  "timestamp_created": "2026-04-19T09:38:44",
  "current_role": "phase0" | "role1" | ...,
  "flow_status": "registered" | "inbound" | "arrived" | ...,
  "synced": true,
  "analyzed": true,
  "rfid_tag_id": "8AEF10C3",
  "device_id": "jetson-01",

  "name": "Erika Schmidt",
  "rank": "Hauptfeldwebel",
  "unit": "3./SanRgt 1",
  "blood_type": "0+",

  "triage": "T2",
  "injuries": ["Kopfverletzung", "Nasenbluten"],
  "mechanism": "Splitterverletzung",
  "medications": [...],
  "treatments": [...],
  "vitals": {
    "pulse": 110,
    "bp": "120/80",
    "resp_rate": 18,
    "spo2": 94,
    "gcs": 15
  },

  "template_type": "9liner",
  "nine_liner": {
    "line1": "UTM-Grid",
    ...
    "line9": "NBC"
  },

  "transcripts": [{"time": "...", "text": "..."}],
  "timeline": [{"event": "registered", "time": "...", "details": "..."}],
  "confidences": {
    "name": 0.95,
    "vitals": {"pulse": 0.89, "spo2": 0.85}
  }
}
```

Patient-Flow-States

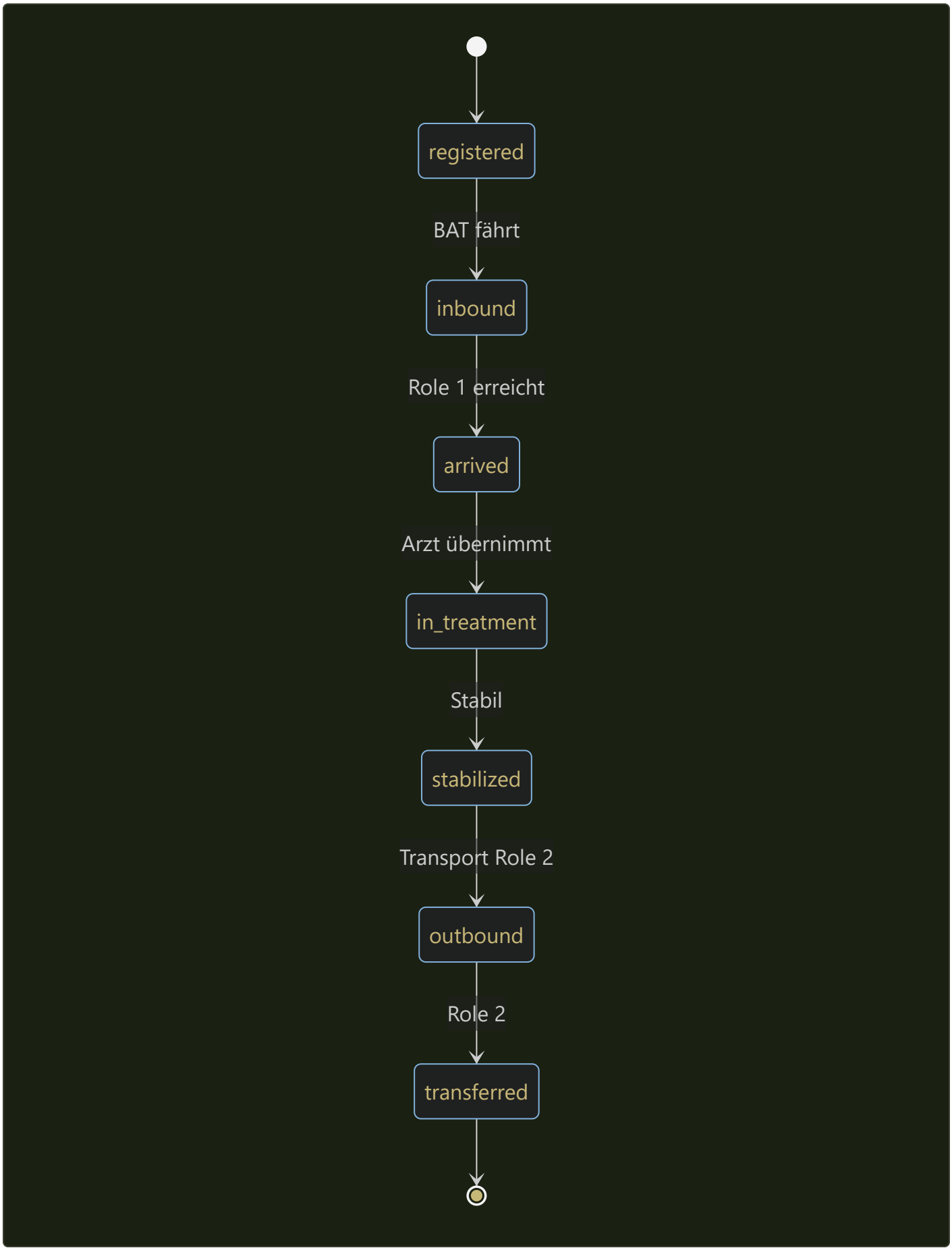


Abb. 3 — Patient-Flow-State-Machine

6. Sicherheitsarchitektur

Verschlüsselte Kommunikation

Kommunikation zwischen Jetson und Surface läuft ausschließlich über das **Tailscale-Mesh** auf **WireGuard**-Basis.

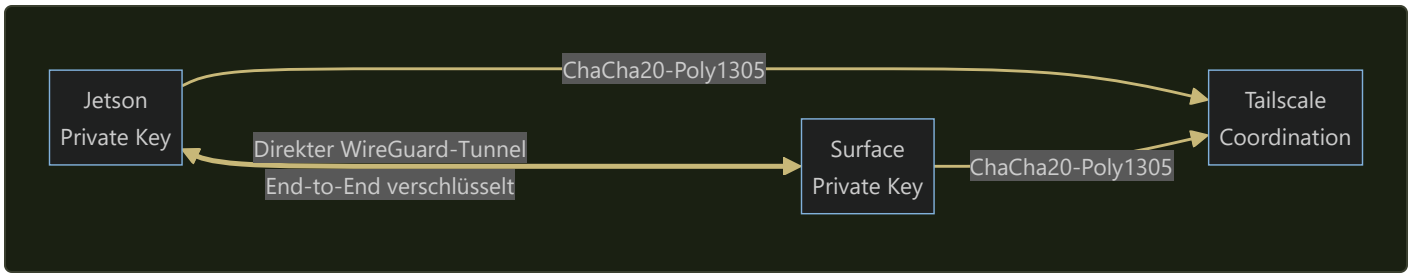


Abb. 4 — E2E-Verschlüsselung: Tailscale orchestriert nur Public-Keys, keine Daten

Kryptographische Primitive (WireGuard-Standard)

FUNKTION	ALGORITHMUS	BIT-LÄNGE
Key Exchange	Curve25519 ECDH	256
Symmetrische Verschlüsselung	ChaCha20	256
Authentifizierte Verschlüsselung	Poly1305	128
Hash / MAC	Blake2s	256

ZERO-TRUST

Tailscale sieht nur Peer-IPs und Public Keys — **keine Payload-Daten**. Selbst Tailscale Inc. kann die Patientenkommunikation nicht mitlesen.

RFID-Sicherheit (Bewusste Einschränkung)

MIFARE Classic Crypto1 wurde 2007 per Brute-Force geknackt. SAFIR behandelt die Karte daher als **reine ID-Referenz**:

- ✓ Patient-ID und Basisdaten (Name, Triage, Vitals) auf der Karte für Offline-Lesbarkeit
- ✗ Transkripte, 9-Liner-Details, Operator-Namen NICHT auf der Karte
- ✓ Backend-Store mit Tailscale-Verschlüsselung als Wahrheitsquelle

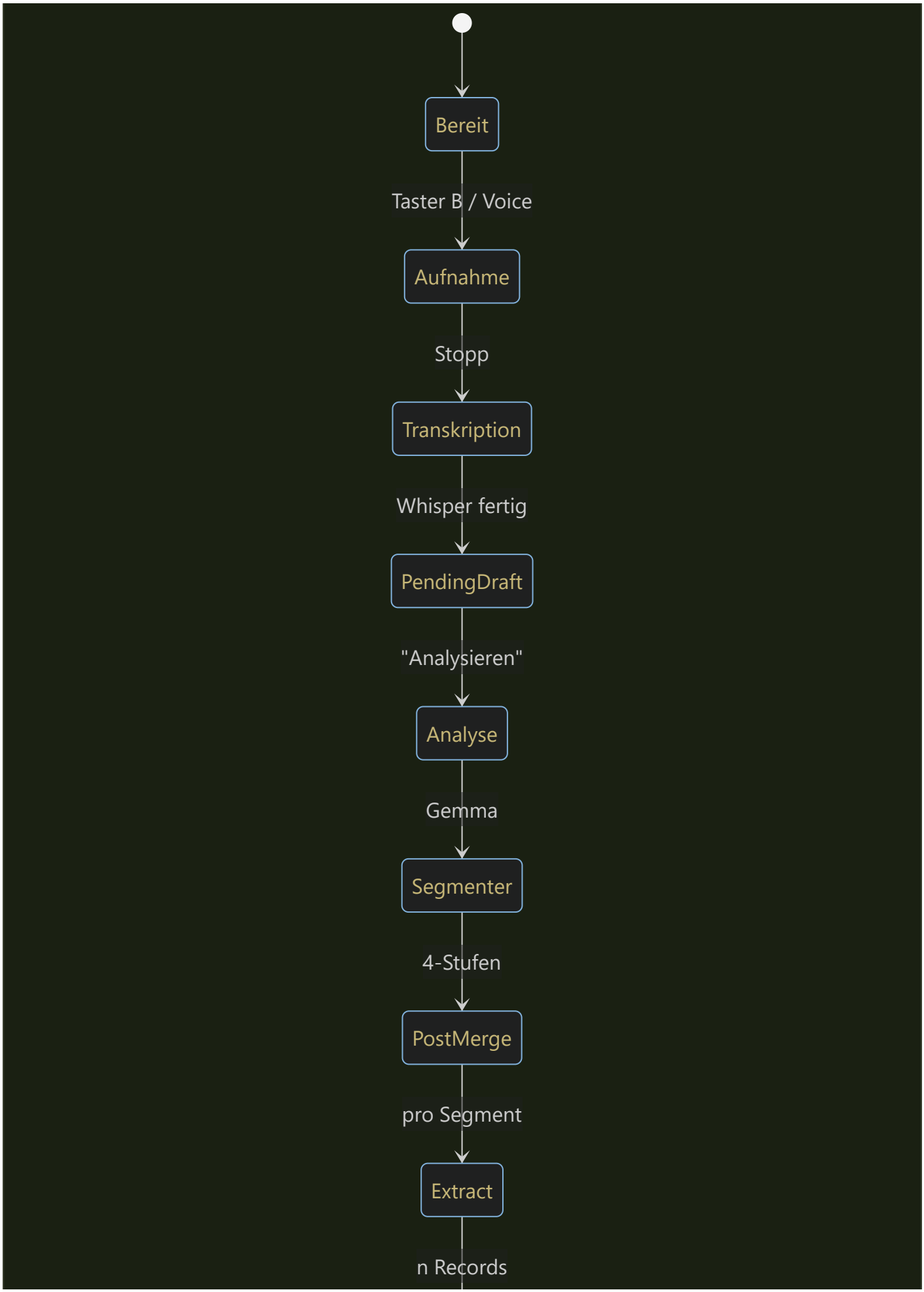
Authentifizierung via RFID-Operator-Karte

Jeder Sanitäter hat eine persönliche Karte, UID in `config.json → rfid.operators`. Auflegen → Login mit TTS-Begrüßung. Rollen-System:

ROLLE	ERLAUBNISSE
`bat_soldat_1`	patient_create, patient_update, rfid_write_patient
`arzt`	+ patient_delete, rfid_overwrite_patient

7. Workflows

Multi-Patient-Diktat


Bereit
Taster B / Voice
Aufnahme
Stopp
Transkription
Whisper fertig
PendingDraft
"Analysieren"
Analyse
Gemma
Segmenter
4-Stufen
PostMerge
pro Segment
Extract
n Records

Post-Merge-Strategie (Defense in Depth)

LLMs sind nicht deterministisch — selbst mit `temperature=0.0`. Vier aufeinanderfolgende Regeln härten das Ergebnis:

1. **Post-Merge 1** — Segmente < 30 chars an vorheriges mergen
2. **Post-Merge 2** — Pronomen-Segmente ("Er hat...", "Sie ist...") gehören zum vorherigen Patient
3. **Post-Merge 3** — Ohne `START_MARKER` UND `PATIENT_MARKER` → zum vorherigen
4. **Post-Merge 4** — Intro-Filter: "Hier spricht..." wird mit dem folgenden echten Patient vereint

RFID-Schreib-Batch

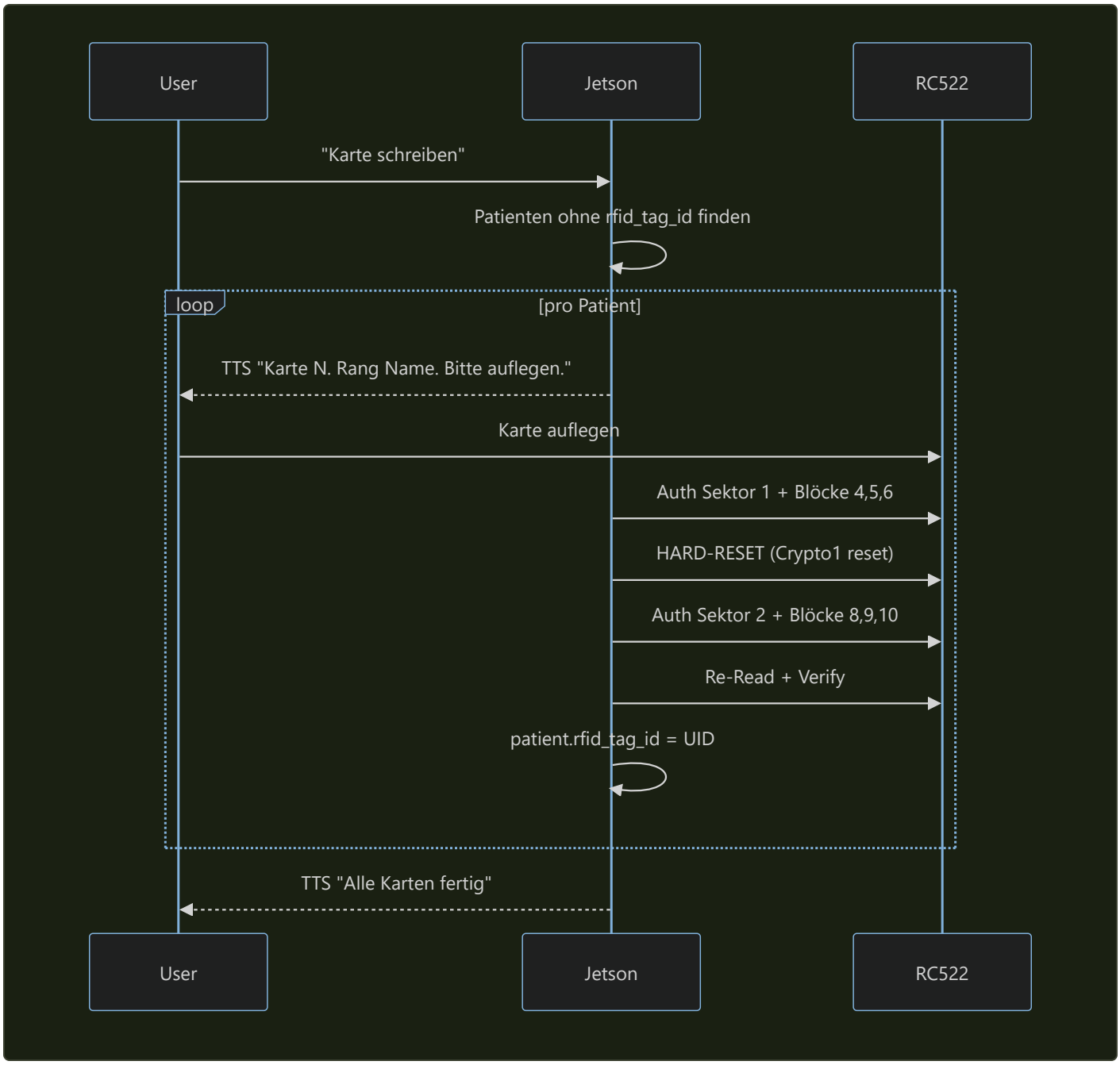


Abb. 5 — RFID-Write mit obligatorischem Hardware-Reset zwischen Sektoren

HARDWARE-RESET KRITISCH

Der RC522-Chip behält nach Sektor-1-Auth einen Crypto1-State. Ohne Reset schlägt Sektor-2-Auth silent fehl und der Name wird nicht geschrieben. Siehe `shared/rfid.py:599-649` für die ausführliche Analyse.

8. API-Referenz

Jetson (Port 8080)

ENDPOINT	METHOD	ZWECK
`/api/status`	GET	System-Status, Modelle, Memory
`/api/patients`	GET	Alle Patient-Records
`/api/pending`	GET	Unanalysierte Transkripte
`/api/recording/start`	POST	Diktat starten
`/api/recording/stop`	POST	Diktat beenden
`/api/analyze/pending`	POST	LLM-Analyse
`/api/rfid/batch`	POST	Batch-Karte-Schreiben
`/api/rfid/erase`	POST	Karte löschen
`/api/sync/all`	POST	Alle Patienten melden
`/ws`	WS	Live-Events Sink+Source

Surface (Port 8080)

ENDPOINT	METHOD	ZWECK
`/api/ingest`	POST	Patient von Jetson empfangen
`/api/position`	POST	BAT-Position aktualisieren
`/api/rfid/clear-tag`	POST	RFID-Zuordnung auflösen
`/api/rfid/lookup`	POST	Manueller UID-Lookup
`/ws`	WS	Live-Events zu Browsern

9. Konfiguration

Via `config.json` auf jedem Gerät. Editierbar über Settings-UI, keine Code-Änderung nötig.

```
{
  "ollama": {
    "url": "http://127.0.0.1:11434",
    "model": "gemma3:4b",
    "num_ctx": 2048,
    "num_gpu": -1,
    "keep_alive": -1
  },
  "backend": {
    "url": "http://100.101.80.64:8080"
  },
  "voice_commands": {
    "record_start": {"triggers": ["aufnahme starten", ...]},
    "rfid_write_patient": {"triggers": ["karte schreiben", ...]},
    ...
  },
  "rfid": {
    "operators": [
      {"uid": "8AEF10C3", "name": "OFA Hugendubel", "role": "arzt"}
    ],
    "roles": {...}
  },
  "security": {
    "lock_idle_seconds": 1800
  },
  "device_id": "jetson-01",
  "unit_name": "BAT Alpha42",
  "rescue_station": {
    "lat": 50.7374,
    "lon": 7.0982,
    "name": "Bonn-Hardthöhe"
  }
}
```

10. Deployment

Jetson Setup

1. JetPack 5.x installieren (Ubuntu 20.04 + CUDA 12.6)
2. Headless: `sudo systemctl set-default multi-user.target`
3. Repo klonen: `git clone git@github.com:Ajintaro/SAFIR.git /home/jetson/cgi-afcea-san`
4. Python-Venv: `python -m venv venv && pip install -r requirements.txt`

5. Ollama + Gemma: `ollama pull gemma3:4b`
6. Vosk, Whisper, Piper nach `models/` entpacken
7. Systemd: `sudo systemctl enable safir.service`
8. Tailscale: `sudo tailscale up --ssh`

Surface Setup

1. Python 3.11+ installieren
2. Repo klonen
3. `pip install -r backend/requirements.txt`
4. Tailscale einrichten
5. Start: `backend\start-surface.cmd` (mit Auto-Reload)
6. Browser: `http://localhost:8080`

11. Performance & Limits

Latenz-Budget

STUFE	TYPISCH	MAX
Whisper-Transkription (pro 25-s-Chunk)	3–5 s	8 s
Gemma Boundary-Segmentation	5–8 s	12 s
Gemma Extract (pro Patient)	10–20 s	40 s
End-to-End (1 Patient)	~30 s	~60 s
Jetson→Surface Sync	< 1 s	3 s
Omnikey-Scan → Modal	< 2 s	3 s

Ressourcen-Nutzung (Swap-Mode)

MODE	WHISPER	GEMMA 3 4B	FREI (VON 7.4 GB)
Idle	0	0	7.4 GB
Recording	1.2 GB	0	6.2 GB
Analyse	0 (unloaded)	4.3 GB	3.1 GB
Analyse fertig	0	0	7.4 GB

Swap-Mode: Vor jeder Analyse wird Whisper ausgelagert, Gemma geladen. Danach umgekehrt. Beide passen nicht gleichzeitig, aber beide werden auch nicht gleichzeitig benötigt.

12. Known Limitations & Roadmap

Kritische Limits

LIMIT	IMPACT	PLAN
Unterbrochene Diktate → Patient verpasst	Arzt ergänzt manuell	Prompt-Beispiel + Confidence-Threshold
MIFARE Classic nicht sicher	Daten auf Karte mitlesbar	Daten minimal halten
Gemma setzt keine Triage	Arzt triagiert	Beabsichtigt
Kein Identity-Provider	User in JSON	LDAP für v3

Roadmap

- **v2.2** — Messe-Hardening A1-A5, Confidence-Badges, Preset-Szenarien, Backup-Jetson
- **v3.0** — 9-Liner MEDEVAC Full-Flow, TCCC-Card, Multi-Station (Role 2+3), PostgreSQL
- **v4.0** — SitaWare-Integration, SATCOM-Modus, Voice-Biometrie, Cross-Border

---

Kontakt

CGI Deutschland · AFCEA-Messe 2026

Repository: `github.com/Ajintaro/SAFIR`